Doc. no: P1133R0
Audience: LEWG
Date: 2018-06-21
Reply-To: Vinnie Falco (vinnie.falco@gmail.com)

Networking TS Associations For Call Wrappers

Contents

Abstract

In [networking.ts] (also called "TS" henceforth) a *CompletionHandler* is a callable object invoked when an asynchronous operation has completed. The TS specifies customization points allowing an *executor*, a *proto-allocator*, or both, to be associated with an instance of a completion handler. Authors of asynchronous algorithms frequently need to bind parameters to a completion handler and submit the resulting call wrapper to another algorithm. Unfortunately when doing so with a lambda, or a forwarding call wrapper such as `std::bind`, customizations associated with the original completion handler are lost. This paper proposes wording which forwards TS associations to call wrappers returned by the standard library.

Introduction

Asynchronous algorithms are started by a call to an *initiating function* which returns immediately. When the operation has completed, the implementation invokes a caller-provided function object called a *completion handler* with the result of the operation expressed as a parameter list. The following code shows calls to initiating functions with various types of completion handlers passed as the last argument.

Using a lambda:

```
async_read([…],
    [](std::error_code ec, std::size_t)
    {
        if(ec)
            std::cerr << ec.message() << "\n";
    });
```

Using a function object:

```
struct my_completion
{
    void operator()(std::error_code, std::size_t);
};

async_read([…], my_completion{});
```

Using a call wrapper:

```
struct session
{
    […]

    void on_read(std::error_code, std::size_t)
    {
        async_read(sock_, buffers_,
            std::bind(
                &session::on_read,
                this,
                std::placeholders::_1,
                std::placeholders::_2));
```

```
    }
};
```

To allow for greater control over memory allocations performed by the implementation or asynchronous algorithms, the TS allows an allocator to be associated with the type of the completion handler. This can be accomplished two ways.

Intrusively, by adding a nested type and member function:

```
struct my_completion
{
    using allocator_type = […]

    allocator_type get_allocator() const noexcept;

    void operator()(std::error_code, std::size_t);
};
```

Or by specializing a class template:

```
namespace std {
namespace experimental {
namespace net {

template<class Allocator>
struct associated_allocator<my_completion>
{
    using type = […]

    static type get(my_completion const& h, Allocator const& a = Allocator()) noexcept;
};

}}}
```

The TS also allows control over the executor used to invoke the completion handler, through customizations similar to the associated allocator.

Intrusively, by adding a nested type and member function:

```
struct my_completion
{
    using executor_type = […]

    executor_type get_executor() const noexcept;

    void operator()(std::error_code, std::size_t);
};
```

Or by specializing a class template:

```
namespace std {
namespace experimental {
namespace net {

template<class Executor>
struct associated_executor<my_completion>
{
    using type = […]

    static type get(my_completion const& h, Executor const& ex = Executor()) noexcept;
};

}}}
```

Problem

Algorithms expressed in terms of other asynchronous algorithms are called *composed operations*. An initiating function constructs the composed operation as a callable function object containing the composed operation state as well as ownership of the caller's completion handler, and then invokes the resulting object. The class below implements a composed operation which reads from a stream asynchronously into a dynamic buffer until a condition is met (the corresponding initiating function is intentionally omitted):

```
template<
    class AsyncReadStream,
    class DynamicBuffer,
    class Condition,
    class Handler>
class read_until_op
{
    AsyncReadStream& stream_;
    DynamicBuffer& buffer_;
    Condition cond_;
    Handler handler_;

public:
    read_until_op(
        AsyncReadStream& stream,
        DynamicBuffer& buffer,
        Condition const& cond,
        Handler handler)
        : stream_(stream)
        , buffer_(buffer)
        , cond_(cond)
        , handler_(std::move(handler))
    {
    }

    void operator()(std::error_code ec, std::size_t bytes_transferred)
    {
        if(! ec)
        {
            if(bytes_transferred > 0)
                buffer_.commit(bytes_transferred);

            if(! cond_())
                return stream_.async_read_some(
                    buffer_.prepare(1024),
                    std::move(*this));
        }

        handler_(ec);
    }
};
```

The implementation above contains subtle but significant defects which have been a common source of bugs for years when using Asio or Boost.Asio. Any allocator or executor associated with `Handler` is not propagated to `read_until_op`, and thus will not be used in the implementation of the call to `async_read_some`. In particular multi-threaded network programs using the code above will experience undefined behavior when the associated executor of a completion handler is a `strand`, which is used to guarantee that handlers are not invoked concurrently. The code above may be fixed by associating the composed operation with the same executor and allocator associated with the final completion handler. The simpler, intrusive method may be used here as we have access to the necessary executor through the supplied stream:

```
template<
    class AsyncReadStream,
    class DynamicBuffer,
    class Condition,
    class Handler>
class read_until_op
{
    AsyncReadStream& stream_;
    DynamicBuffer& buffer_;
    Condition cond_;
    Handler handler_;

public:
    read_until_op(
        AsyncReadStream& stream,
        DynamicBuffer& buffer,
        Condition const& cond,
        Handler handler)
        : stream_(stream)
        , buffer_(buffer)
        , cond_(cond)
        , handler_(std::move(handler))
    {
    }
```

```
    using allocator_type = std::experimental::net::associated_allocator<Handler>

    allocator_type get_allocator() const noexcept
    {
        return std::experimental::net::get_associated_allocator(h_);
    }

    using executor_type = std::experimental::net::associated_executor<
        Handler, decltype(std::declval<AsyncReadStream&>().get_executor())>

    executor_type get_executor() const noexcept
    {
        return std::experimental::net::get_associated_executor(handler_, stream_.get_executor());
    }

    void operator()(std::error_code ec, std::size_t bytes_transferred)
    {
        if(! ec)
        {
            if(bytes_transferred > 0)
                buffer_.commit(bytes_transferred);

            if(! cond_())
                return stream_.async_read_some(
                    buffer_.prepare(1024),
                    std::move(*this));
        }

        handler_(ec);
    }
};
```

The addition of the nested types and member functions give `read_until_op` the same allocator and executor associated with the caller-provided completion handler. These associations are visible to other initiating functions, such as in the call to the stream's `async_read_some`. Unfortunately, the code above still has yet another subtle problem with significant consequences. Consider the case where `cond_()` evaluates to true upon the first invocation. The call to the stream's asynchronous read operation will be skipped, and the handler will be invoked directly with an error code indicating success. This violates the TS requirement that the final completion handler is invoked through the associated executor:

> **13.2.7.12 Execution of completion handler on completion of asynchronous operation [async.reqmts.async.completion]**
>
> If an asynchronous operation completes immediately (that is, within the thread of execution calling the initiating function, and before the initiating function returns), the completion handler shall be submitted for execution as if by performing `ex2.post(std::move(f), alloc2)`. Otherwise, the completion handler shall be submitted for execution as if by performing `ex2.dispatch(std::move(f), alloc2)`.

The TS defines the helper functions `post` and `dispatch` to provide the allocator boilerplate in the executor expressions above. However, executors expect nullary function objects (invocable with no arguments). In order to submit a call to the handler in the code above, it is necessary to use a lambda to create a nullary function which invokes the handler with the bound error code. Use of the lambda, along with a bit of extra code to avoid a double dispatch for the case where the completion handler is invoked as a continuation of the call to `async_read_some` would look thusly:

```
void read_until_op::operator()(
    std::error_code ec, std::size_t bytes_transferred, bool is_continuation = true)
{
    if(! ec)
    {
        if(bytes_transferred > 0)
            buffer_.commit(bytes_transferred);

        if(! cond_())
            return stream_.async_read_some(
                buffer_.prepare(1024),
                std::move(*this));
    }

    if(! is_continuation)
        std::experimental::net::post(
            stream_.get_executor(),
            [self = std::move(*this), ec] { self_.handler_(ec); });
```

```
        else
            handler_(ec);
    }
```

Astute observers may wonder why the handler is called directly when the composed operation is invoked as a continuation of the call to `async_read_some` instead of using the associated executor's `dispatch` function. The reason is that we are guaranteed that the composed operation was already invoked through the associated exector's dispatch function, because the implementation of `async_read_some` must meet the requirements of 13.2.7.12 [async.reqmts.async.completion].

Readers without a deep understanding of Asio or [networking.ts] may not realize that the code above contains another defect. It suffers from the same problem found in the original implementation. The lambda type does not have the same allocator and executor associations as the final completion handler, thus violating contracts.

Having the lambda forward the associated allocator and executor of the contained completion handler requires additional syntax and change to the language. This paper does not propose such a language change, as none of the possible changes explored by the author look palatable in the slightest.

Since the intent of the statement in question is to bind arguments to a callable, to produce a new callable, a logical alternative is to consider using `std::bind`, which looks like this:

```
std::experimental::net::post(
    stream_.get_executor(),
    std::bind(std::move(*this), ec));
```

However, once again the code contains a defect! It does not solve the problem, because the call wrapper returned by `bind` does not forward the necessary associations. But unlike the lambda, in this case the type is emitted by the library. Therefore, the library can provide the specializations for the call wrapper.

Solution

The solution proposed in this paper is to specialize the [networking.ts] class templates `associated_allocator` and `associated_executor` for selected call wrappers returned by standard library functions. Each set of specializations will simply forward the allocator and executor associated with the wrapped function object to the call wrapper. We note that there is at least one paper in flight (P0356R1) which adds new call wrappers to the language, `bind_front` and `bind_back`. They will need a similar treatment.

Here, some alternatives are explored and exploratory questions are answered:

- **Can't we use a lambda instead of a call wrapper?**

  The problem with using a lambda expression to bind arguments to a completion handler is the implicit type-erasing of the captured completion handler. The anonymous type of the lambda expression emitted by the compiler is not part of any specialization of the TS class templates `associated_allocator` and `associated_executor`. Wording to address this in lambdas encounters strong headwinds:

  - The language specification now has a coupling to [networking.ts].
  - The language has no means to express that a lambda should inherit the associated allocator and executor of a captured completion handler.

  For these reasons, this paper does not discuss solutions which require changes to the language itself.

- **Shouldn't this apply to most call wrappers not just `std::bind`?**

  In a nutshell, yes. However, specializations for type-erased call wrappers (e.g. `std::function`) are unimplementable.

- **How often is a TS-aware call wrapper actually needed?**

  Authors of composed operations will almost always need the facility described in this paper. We note that Asio[1], networking-ts-impl[2], and Beast[3] each contain their own implementations of call wrappers which forward the associated allocator and executor. Beast's interface is public, while the others are implementation details. More generally, the requirements specified in [networking.ts] 13.2.7.12 are difficult to meet without a TS-aware call wrapper. A survey of open source projects on GitHub shows that users repeatedly mistake losing the allocator and executor association of completion handlers by naive use of lambdas or call wrappers.

- **Why did you design allocator and executor associations this way?**

Note that the author of this paper is not the author and architect of [networking.ts]. That person is Christopher Kohlhoff, who may be reached by email. While this author can answer some questions about design choices, ultimately it is the principal architect of the TS who can give accurate and comprehensive answers to question regarding design decisions.

- **Is there a better way to implement these customization points?**

  Changing the method used by the TS to specify the associated allocator and executor for completion handlers is out of scope for this paper, but could be the subject of another paper. However, there is evidence to suggest that such a paper will not be successful. The allocator and executor customizations have the advantage of 13 years of existing practice in Boost, in other open source projects, and in commercial settings. They have gone through a couple of iterations along the way, in response to real world issues. These customization points are not just associated types, but also algorithms to obtain the correct instance of the type, which also allows the caller to specify a fallback for the case where a selected default is needed. This author has not been able to come up with another solution which both maintains the expressive power of the TS customization points, and exhibits the same or better level of simplicity and elegance. Perhaps someone else will have a eureka moment and discover a better way, but it seems unlikely.

- **Can we make these customization points less error-prone?**

  From the author's experience and the visible cascade of problems with the example code in this paper, writing initiating functions and composed operations correctly requires a demanding amount of experience and skill. This is especially true when considering that such algorithms must also implicitly exhibit correct behavior in multi-threaded environments. This is accomplished by achieving a deep and thorough understanding of Asio or [networking.ts] and applying that understanding to code. The author and contributors to Boost.Beast have explored library solutions to provide some correct boilerplate and higher level idioms to users in order to make authoring composed operations easier to write. A little bit of progress has been made on this front, but comprehensive improvements have been elusive. We suspect that grand unifying solutions simply do not exist, and that the level of complexity is inherent to the domain.

- **Should this be spelled `std::experimental::net::bind`?**

  That also solves the problem, but introduce two new ones:

  - Standard libraries now have separate bind implementations which differ only in that one of them is TS-aware, and the other isn't.
  - The guidance to users becomes "use this call wrapper, unless you are wrapping a completion handler, in which case use this other call wrapper."

  We find this objectively worse than leveraging the call wrappers which already exist in the standard library.

- **How about we remove `std::bind` and add `std::experimental::net::bind`?**

  Functions like `std::bind` which return call wrappers are part of the standard library today, and removing them is out of scope for this paper, so we do not propose that here.

- **But `std::bind` is bad, I will only accept its deprecation and removal!**

  Technically that isn't a question. We note that the improvements suggested in this paper are by no means a show-stopper. [networking.ts] can go on without it, and we could always add this in a future revision of the standard if desired. Authors of composed operations can write their own call wrapper, or they can use the one provided in Boost.Beast which is rapidly gaining popularity for its approach of compatibility with [networking.ts] and established practice. Regardless, a non-zero fraction of users will reach for `std::bind` when authoring library code that accepts instances of completion handlers. Without these changes, those users will produce code which is certain to be incorrect. Therefore, for as long as the standard library provides functions which return call wrappers, it is prudent to ensure that those call wrappers are well integrated with [networking.ts].

Wording

This wording is relative to N4734.

1. Modify 13.5 [async.assoc.alloc], as indicated:

   […]

   -3- The implementation provides partial specializations of `associated_allocator` for the forwarding call wrapper types returned by `std::bind` and `std::ref`. If `g` of type `G` is a forwarding call wrapper with target object `fd` of type `FD`, and `a` is a

ProtoAllocator of type `PA` then:

- `associated_allocator<G, PA>::allocator_type` is `associated_allocator<FD, PA>::allocator_type`
- `associated_allocator<G, PA>::get(g)` returns `associated_allocator<FD, PA>::get(fd)`
- `associated_allocator<G, PA>::get(g, a)` returns `associated_allocator<FD, PA>::get(fd, a)`

2. Modify 13.12 [async.assoc.exec], as indicated:

[…]

-3- The implementation provides partial specializations of `associated_executor` for the forwarding call wrapper types returned by `std::bind` and `std::ref`. If `g` of type `G` is a forwarding call wrapper with target object `fd` of type `FD`, and `e` is a *Executor* of type `E` then:

- `associated_executor<G, E>::executor_type` is `associated_executor<FD, E>::executor_type`
- `associated_executor<G, E>::get(g)` returns `associated_executor<FD, E>::get(fd)`
- `associated_executor<G, E>::get(g, e)` returns `associated_executor<FD, E>::get(fd, e)`

References

[1] https://github.com/boostorg/asio/blob/fbe86d86b1ac53e40444e5af03ca4a6c74c33bda/include/boost/asio/detail/bind_handler.hpp#L32

[2] https://github.com/chriskohlhoff/networking-ts-impl/blob/3524b4408d26a67af683bfd2aad6b0b6b5684b36/include/experimental/__net_ts/detail/bind_handler.hpp#L34

[3] https://www.boost.org/doc/libs/1_67_0/libs/beast/doc/html/beast/ref/boost__beast__bind_handler.html